

Cet après-midi, vous mettez en pratique les notions du matin sur trois situations tirées du quotidien d'un analyste en sécurité. Chaque défi dure **environ 25 minutes**.

Règles du jeu

- Vous codez **vous-mêmes, sans IA** : ni ChatGPT, ni Claude, ni autocomplétion Copilot. Désactivez-la dans VS Code.
- La documentation Python est autorisée, et même encouragée.
- Un fichier par défi : `defi1.py` , `defi2.py` et `defi3.py` , dans votre dossier de travail avec son `venv` activé.
- **Validation** : quand un défi est terminé, levez la main. Vous exécutez votre code devant moi, puis je vous pose une question sur votre code.
- **Notation** : non fait (0), partiel (1) ou réussi (2). Les défis comptent pour 10 % de la note du module.
- **Parcours confirmé** : validez ces trois défis, puis passez aux défis avancés.

Défi 1

Politique de mots de passe

Défi 2

Détection de brute force

Défi 3

Générateur de mots de passe



Politique de mots de passe

≈ 25 min

Conditions · boucles · fonctions

Audit

Vous auditez la politique de sécurité d'une entreprise. Son application interne accepte n'importe quel mot de passe : vous devez écrire la fonction qui fera respecter les règles suivantes, inspirées des recommandations de l'ANSSI.

Règle	Message d'erreur si elle n'est pas respectée
12 caractères minimum	au moins 12 caractères
Au moins une majuscule	au moins une majuscule
Au moins une minuscule	au moins une minuscule
Au moins un chiffre	au moins un chiffre
Au moins un caractère spécial (ni lettre, ni chiffre)	au moins un caractère spécial
Absent de la liste des mots de passe courants, sans tenir compte de la casse	ne doit pas être un mot de passe courant

Votre mission

1. Copiez la liste des mots de passe courants ci-dessous en haut de `defi1.py`.
2. Écrivez une fonction `check_password(password)` qui renvoie **un tuple** de deux valeurs : `True` ou `False`, et la liste des messages d'erreur (vide si le mot de passe est valide).
3. Testez-la avec les exemples attendus.

Données à copier

```
COMMON_PASSWORDS = [  
    "123456", "password", "azerty", "qwerty", "motdepasse",  
    "admin", "letmein", "iloveyou", "000000", "azertyuiop",  
    "Password123!", "Azerty123456!",  
]
```

Exemples attendus

Résultats attendus

```
print(check_password("Tr0ub4dour&Co"))
# (True, [])

print(check_password("Azerty123456!"))
# (False, ['ne doit pas être un mot de passe courant'])

print(check_password("azerty"))
# (False, ['au moins 12 caractères', 'au moins une majuscule',
#         'au moins un chiffre', 'au moins un caractère spécial',
#         'ne doit pas être un mot de passe courant'])

print(check_password("correct horse battery staple"))
# (False, ['au moins une majuscule', 'au moins un chiffre'])
```

Indices

- Une chaîne se parcourt caractère par caractère avec une boucle `for`.
- Cherchez dans la documentation les méthodes de chaîne `isupper()`, `islower()`, `isdigit()` et `isalnum()`.
- Pour ignorer la casse, comparez tout en minuscules.

Pour valider, montrez-moi...

- les quatre exemples ci-dessus, qui donnent exactement ces résultats ;
- un mot de passe de votre choix, valide.

Bonus

- Demandez un mot de passe à l'utilisateur **jusqu'à ce qu'il soit valide**, en affichant les règles non respectées à chaque essai.
- Masquez la saisie avec `getpass`, comme le fait `sudo`.
- Réfléchissez : « correct horse battery staple » est long et difficile à deviner, et pourtant il est refusé. Ces règles sont-elles vraiment les bonnes ?

2

Détection de brute force

≈ 25 min

Chaînes · dictionnaires · boucles

SOC

Vous êtes analyste dans un SOC. Un serveur web exposé sur Internet reçoit des connexions SSH suspectes. Voici un extrait de son journal d'authentification (`/var/log/auth.log`). Votre mission : repérer les adresses IP qui tentent de deviner des mots de passe par **brute force**, comme le fait l'outil fail2ban.

Données à copier – journal d'authentification

```
AUTH_LOG = [
    "Oct 7 09:12:03 srv-web sshd[2101]: Accepted password for alice from 192.0.2.10 port 50122 ssh2",
    "Oct 7 09:40:17 srv-web sshd[2144]: Failed password for bob from 192.0.2.25 port 50388 ssh2",
    "Oct 7 09:40:29 srv-web sshd[2144]: Accepted password for bob from 192.0.2.25 port 50388 ssh2",
    "Oct 7 10:02:11 srv-web sshd[2210]: Failed password for root from 203.0.113.45 port 51122 ssh2",
    "Oct 7 10:02:14 srv-web sshd[2212]: Failed password for root from 203.0.113.45 port 51130 ssh2",
    "Oct 7 10:02:18 srv-web sshd[2215]: Failed password for invalid user admin from 203.0.113.45 port 51141 ssh2",
    "Oct 7 10:02:21 srv-web sshd[2218]: Failed password for invalid user test from 203.0.113.45 port 51150 ssh2",
    "Oct 7 10:02:25 srv-web sshd[2221]: Failed password for alice from 203.0.113.45 port 51162 ssh2",
    "Oct 7 10:02:29 srv-web sshd[2224]: Failed password for alice from 203.0.113.45 port 51170 ssh2",
    "Oct 7 10:02:33 srv-web sshd[2227]: Failed password for alice from 203.0.113.45 port 51178 ssh2",
    "Oct 7 10:02:36 srv-web sshd[2230]: Accepted password for alice from 203.0.113.45 port 51185 ssh2",
    "Oct 7 10:02:37 srv-web sshd[2230]: pam_unix(sshd:session): session opened for user alice by (uid=0)",
    "Oct 7 10:15:40 srv-web sshd[2301]: Failed password for invalid user oracle from 198.51.100.23 port 40022 ssh2",
    "Oct 7 10:15:44 srv-web sshd[2303]: Failed password for invalid user postgres from 198.51.100.23 port 40031 ssh2",
    "Oct 7 10:15:47 srv-web sshd[2305]: Failed password for root from 198.51.100.23 port 40040 ssh2",
    "Oct 7 10:15:51 srv-web sshd[2307]: Failed password for invalid user ubuntu from 198.51.100.23 port 40052 ssh2",
    "Oct 7 10:15:55 srv-web sshd[2309]: Failed password for invalid user pi from 198.51.100.23 port 40060 ssh2",
    "Oct 7 10:16:02 srv-web sshd[2311]: Connection closed by 198.51.100.23 port 40071 [preauth]",
    "Oct 7 11:30:08 srv-web sshd[2402]: Failed password for carol from 198.51.100.77 port 42210 ssh2",
    "Oct 7 11:30:15 srv-web sshd[2404]: Failed password for carol from 198.51.100.77 port 42218 ssh2",
    "Oct 7 11:31:02 srv-web sshd[2410]: Accepted password for carol from 198.51.100.77 port 42230 ssh2",
]
```

Votre mission

1. Écrivez `count_failures(lines)`, qui renvoie un **dictionnaire** associant à chaque adresse IP son nombre de lignes `Failed password`.
2. Écrivez `detect_brute_force(lines, threshold=5)`, qui renvoie la **liste** des IP ayant **au moins** `threshold` échecs.
3. Écrivez `print_report(lines, threshold=5)`, qui affiche une alerte par IP détectée.

Exemples attendus

Résultats attendus

```
print(count_failures(AUTH_LOG))
# {'192.0.2.25': 1, '203.0.113.45': 7, '198.51.100.23': 5, '198.51.100.77': 2}

print(detect_brute_force(AUTH_LOG))
# ['203.0.113.45', '198.51.100.23']

print(detect_brute_force(AUTH_LOG, threshold=2))
# ['203.0.113.45', '198.51.100.23', '198.51.100.77']

print_report(AUTH_LOG)
# [ALERTE] 203.0.113.45 : 7 échecs d'authentification
# [ALERTE] 198.51.100.23 : 5 échecs d'authentification
```

Indices

- Attention : toutes les lignes n'ont pas le même nombre de mots (« invalid user »). L'adresse IP est toujours **le mot qui suit** `from`.
- La méthode de liste `index()` donne la position d'un élément : `words.index("from")`.
- Revoyez la diapo « Compter avec un dictionnaire » du cours et la méthode `get()`.

Pour valider, montrez-moi...

- les quatre exemples ci-dessus, qui donnent exactement ces résultats ;
- le rapport avec un seuil de votre choix.

Bonus

- Pour chaque IP, affichez l'**ensemble des comptes visés** (`root` , `admin` ...). Quelle structure de données choisir ?
- Une IP détectée a-t-elle **fini par réussir à se connecter** ? Si oui, affichez une alerte critique : le compte concerné est probablement compromis.

3

Générateur de mots de passe

≈ 25 min

Modules · fonctions · exceptions

Cryptographie

Maintenant que l'application vérifie les mots de passe, elle doit aussi pouvoir en **proposer** de robustes à ses utilisateurs. Un générateur de secrets doit être imprévisible : vous utiliserez le module `secrets`, et surtout pas `random`.

Votre mission

1. Écrivez `generate_password(length=16)`, qui renvoie un mot de passe aléatoire de `length` caractères.
2. Le mot de passe contient **toujours** au moins une majuscule, une minuscule, un chiffre et un caractère spécial, et ces caractères ne sont pas toujours aux mêmes positions.
3. Si `length` est inférieur à 12, la fonction lève une `ValueError`.
4. Importez `check_password` depuis votre `defi1.py` et vérifiez que **1 000 mots de passe générés** sont tous valides.

Exemples attendus

Résultats attendus (les mots de passe changent à chaque exécution)

```
print(generate_password())
# par exemple : 45h#0_$APXE[QLje

print(generate_password(24))
# par exemple : {U/7/lzny-b{MJb[-BlagCX(

generate_password(8)
# ValueError: longueur minimale : 12

# Vérification avec le défi 1
# Mots de passe invalides sur 1000 : 0
```

Indices

- Le module `string` fournit `string.ascii_uppercase`, `string.ascii_lowercase` et `string.digits`. Choisissez vous-même vos caractères spéciaux.
- `secrets.choice(sequence)` tire un élément au hasard. Commencez par un caractère de chaque catégorie, puis complétez.
- Pour mélanger une liste de manière sûre : `secrets.SystemRandom().shuffle(liste)`. Pour recoller une liste de caractères : `"".join(liste)`.
- `from defi1 import check_password` fonctionne si les deux fichiers sont dans le même dossier.

Pour valider, montrez-moi...

- trois mots de passe générés, de longueurs différentes ;
- l'erreur levée pour une longueur de 8 ;
- la vérification des 1 000 mots de passe avec votre défi 1.

Bonus

- Écrivez `generate_passphrase(n_words=5, separator="-")` , qui assemble des mots tirés au hasard dans une liste (`volcan-radar-hibou-quartz-sabre`), à la manière de la méthode **Diceware**.
- Calculez l'**entropie** en bits de vos secrets : `nombre_de_symboles × log2(taille_de_l'alphabet)` , avec `math.log2()` . Comparez un mot de passe de 16 caractères avec une phrase de 5 mots tirés d'une liste de 30 mots, puis de 7 776 mots (la liste Diceware officielle). Que concluez-vous ?